

The Nimbus Cipher

Alexis Warner Machado

Gauss Informatica, Brazil; email: alexis@brfree.com.br

Oct/03/2000

Abstract. Nimbus is an extremely simple block cipher. Consists of five rounds, each one using only three primitives: multiplication (mod 2^{64}), exclusive-or and a bit-reversal operation. Presents non-linear characteristic and excellent statistical properties, like avalanche effect. Due to its simplicity, the algorithm is very easy to understand, implement and modify.

1 Algorithm Description

The cipher construction steps will be explained in a very intuitive way, avoiding mathematical details.

The round function was inspired on the linear congruential random number generator (LCG). The LCG is defined by

$$y = a \cdot x + b \pmod{m} \quad (1)$$

Where all numbers are integers. Numbers a and b are constant coefficients.

After calculating y from x , we can assign y to x and do the calculus again. This iteration generates a random number sequence.

Now consider a and b secret numbers. Given two pairs (x, y) we can easily find a and b . To make this more difficult we replace the addition with exclusive-or ($\oplus$):

$$y = a \cdot x \oplus b \pmod{m} \quad (2)$$

Now mod m applies only to the multiplication.

Multiplication mod m is very time consuming if m is not a power of 2. Considering 64-bit integer operands, the obvious choice is $m = 2^{64}$:

$$y = a \cdot x \oplus b \pmod{2^{64}} \quad (3)$$

A problem with multiplication mod 2^{64} is that modifying bit n of a multiplicand doesn't affect the result bits lower than n . We want any result bit presenting 50% probability of change. This is known as strict avalanche. To obtain this effect we introduce a bit-reversal (or mirroring) function g . This function swaps bit n with bit $63-n$ for $n = 0..31$. Our equation becomes

$$y = g(a \cdot x) \oplus b \pmod{2^{64}} \quad (4)$$

Note that more than one round is necessary to achieve the avalanche effect.

If a is odd the multiplicative inverse $(\text{mod } 2^{64})$ a' exists. Using the fact that $g(g(x)) = x$ and $b \oplus b = 0$, we can invert equation (4):

$$x = a' \cdot g(b \oplus y) \pmod{2^{64}} \quad (5)$$

The encryption process will be the iteration of five rounds like (5):

$$\begin{aligned} y &\leftarrow k_0 \cdot g(k_5 \oplus y) \\ y &\leftarrow k_1 \cdot g(k_6 \oplus y) \\ y &\leftarrow k_2 \cdot g(k_7 \oplus y) \\ y &\leftarrow k_3 \cdot g(k_8 \oplus y) \\ y &\leftarrow k_4 \cdot g(k_9 \oplus y) \end{aligned} \quad (6)$$

Since all variables involved are 64-bit integers, ‘mod 2^{64} ’ may be considered an implicit operation (all overflow bits of multiplication are discarded).

If we initialize y with two identical integers, obviously we will get the same output. But for different processors, “identical” integers may be different (reversed) byte sequences. When initializing y with a plaintext block we must guarantee that different processors will “see” the same integer, so we consider the first plaintext byte to be the least significant of integer y (little-endian convention).

The sequence k_j ($j = 0 \dots 9$) is a derived key. Elements $k_0 \dots k_4$ must be odd to allow multiplicative inverses $(\text{mod } 2^{64})$. This vector is expected to be a random bit sequence (except the first bit of odd elements). This requirement eliminates many (weak) keys from 2^{635} total (first bit of the 5 odd elements are fixed), but can leave more than 2^{128} . The key derivation is explained in the next section.

The decryption process will be the iteration of five rounds like (4):

$$\begin{aligned} x &\leftarrow g(k_4' \cdot x) \oplus k_9 \\ x &\leftarrow g(k_3' \cdot x) \oplus k_8 \\ x &\leftarrow g(k_2' \cdot x) \oplus k_7 \\ x &\leftarrow g(k_1' \cdot x) \oplus k_6 \\ x &\leftarrow g(k_0' \cdot x) \oplus k_5 \end{aligned} \quad (7)$$

Where x is initialized with the ciphertext block and $k_0' \dots k_4'$ are multiplicative inverses $(\text{mod } 2^{64})$ of $k_0 \dots k_4$.

The same little-endian convention explained for equations (6) must be applied to x .

Note: The 64-bit integer (block) size was chosen for multiplication performance on 64-bit processors. For other sizes (128, 256, ...), the algorithm is the same. The developer may use a compiler that supports these larger integers or implement functions

(multiplication, exclusive-or) for them. Trivial adjustments must be done on bit-reversal and key setup functions.

2 The Key Setup

The integer sequence k_j ($j = 0 \dots n$) is derived from another sequence s_i ($i = 0 \dots m$), where n is odd and $m < n$. The integers k_j and s_i have the same bit size of the block transformed by the cipher.

First we define an auxiliary function “Nimbus_fixedkey” applying equations (6) with a fixed key ($k_0 \dots k_9$).

In the reference implementation, the fixed key is the fractional part of π hexadecimal expansion:

```
243F6A8885A308D3, 13198A2E03707345, A4093822299F31D1,
082EFA98EC4E6C89, 452821E638D01377, BE5466CF34E90C6C,
C0AC29B7C97C50DD, 3F84D5B5B5470917, 9216D5D98979FB1B,
D1310BA698DFB5AC
```

Where the first five integers are forced to be odd.

The pseudo-code to generate k_j from s_i is

```
For j = 0 to n do  $k_j \leftarrow 0$  // k initialization
```

```
For i = 0 to m do
```

```
   $x \leftarrow s_i \oplus \text{Nimbus\_fixedkey}(s_i)$  (8)
```

```
  For j = 0 to n do
```

```
     $x \leftarrow \text{Nimbus\_fixedkey}(x)$  (9)
```

```
     $k_j \leftarrow x \oplus \text{Nimbus\_fixedkey}(x + k_j)$  (10)
```

```
  End For
```

```
End For
```

```
For j = 0 to  $n \div 2$  do  $k_j \leftarrow k_j$  or 1 // Forcing  $k_0 \dots k_{n \div 2}$  to be odd
```

Where $\div$ represents integer division.

Note that, in (8), is hard to find s_i that gives an specific initial value for x . Similarly, in (10), is difficult to find x for a given left side k_j .

The same little-endian convention explained for equations (6) must be applied to s_i .

3 Multiplicative Inverses

In equations (7) we need the multiplicative inverses of $k_0 \dots k_4$. This calculation could be done by the extended Euclid algorithm. But there is a better way for inverses in $Z(\alpha^n)$, where α is an integer and n is a power of 2.

Let

$$\text{i) } \varphi(\mathbf{x}, \mathbf{y}) \text{ be the multiplicative inverse of } \mathbf{x} \text{ in } Z(\mathbf{y}). \quad (11)$$

$$\text{This means: } w = \varphi(x, y) \Leftrightarrow x \cdot w \equiv 1 \pmod{y}$$

$$\text{ii) } \mathbf{r} \text{ be the remainder of an integer } \mathbf{a} \text{ divided by } \mathbf{m}. \quad (12)$$

Then we can show that

$$\varphi(a, m^2) = (2 - a \cdot \varphi(r, m)) \cdot \varphi(r, m) \quad (13)$$

Demonstration :

Representing $\varphi(\mathbf{r}, \mathbf{m})$ by $\mathbf{r}'$ and the right side of (13) by $\mathbf{b}$, we can write

$$r \cdot r' \equiv 1 \pmod{m} \quad (14)$$

and

$$b = (2 - a \cdot r') \cdot r'$$

Multiplying the two sides by $\mathbf{a}$

$$a \cdot b = (2 - a \cdot r') \cdot a \cdot r' \quad (15)$$

From (12)

$$a \equiv r \pmod{m} \Rightarrow a \cdot r' \equiv r \cdot r' \pmod{m}$$

and using (14)

$$a \cdot r' \equiv 1 \pmod{m} \Rightarrow a \cdot r' = 1 + pm \quad (16)$$

for some integer $\mathbf{p}$. Substituting (16) in (15)

$$\begin{aligned} a \cdot b &= (2 - 1 - pm)(1 + pm) = (1 - pm)(1 + pm) = 1 - p^2 m^2 \Rightarrow \\ a \cdot b &\equiv 1 \pmod{m^2} \Rightarrow \\ \varphi(a, m^2) &= b \end{aligned}$$

Note that equation (13) reduce the problem to a similar one, but using the module square root. For a 2^{64} initial module the algorithm need 6 steps.

4 Security

4.1 Linear and Interpolation Cryptanalysis

The combination of modular multiplication with bit-reversal can't be approximated by simple algebraic structures like polynomials. So Nimbus can't be attacked by linear [1,13] and interpolation [6] methods.

4.2 Differential Cryptanalysis

Output block high/low difference probability based on input block lower bits difference needs more rounds to vanish than for higher bits. After the 3rd round, the combination of modular multiplication with bit-reversal eliminate effects of plaintext differentials.

Based on this fact, Fluhrer [12] could mount an interesting $O(2^{64})$ chosen-plaintext truncated-differential attack with four rounds. There is no obvious way to extend it to five rounds with complexity lower than $O(2^{128})$.

4.3 Impossible Differentials and Saturation

Impossible differentials [3, 4] is used to attack IDEA that uses modular multiplications after block partition. Since Nimbus does not partition the data in smaller blocks as in IDEA, I don't see how impossible differential attacks could be mounted against it. Saturation attacks [11] may not be feasible for the same reason.

4.4 Related Keys

The key setup procedure (section 2) generates a sequence k_j that can't be distinguished from a random bit sequence. Sequences k_j derived from different sequences s_i have no statistically detectable correlations, even for highly correlated s_i sequences.

4.5 Timing Attacks

If an optimized multiplication instruction/algorithm is used by the processor/compiler, a timing attack [9] could be used to recover the key. To avoid this kind of attack we could use

- a) Non-optimized multiplication instruction/algorithm. This is common in 64-bit processors and 32-bit compilers.
- b) If the optimized multiplication time depends only on the number of 1's in the multiplicand bits, we can do an additional dummy multiplication using the multiplicands complement.

4.6 Weak Keys

Known weak (derived) keys are those with long runs (> 16) of identical bit values. The key setup procedure (section 2) generates a sequence k_j that can't be distinguished from a random bit sequence, so this situation is very unlikely to occur.

5 Advantages

The main advantage of this algorithm is simplicity. Implementation and improvements can be done by anyone with minimal programming skills. Furthermore, low complexity may reduce efforts to prove cipher strength and weakness.

6 Absence of Hidden Weaknesses

The cipher have **no** backdoors or deliberately inserted weakness.

7 Performance

Algorithm performance, in CBC mode, **without assembly optimization**:

Block size	: 8 bytes = 64 bits
Data encrypted	: 2^{20} blocks = 8 Mbytes
Processor	: Pentium 166 MHz
Memory	: 64 MB RAM
Compiler	: Borland C++ Builder 3.0
Key setup	: 6730 keys/s
Encryption	: 20 Mbit/s = 66 cycles/byte = 528 cycles/block

The bit reversal of all bytes are stored in a table. To reverse an 8 byte integer we need 8 fetches from this table.

In the Itanium 64-bit processor [13], we can roughly estimate the round ($y \leftarrow k_i \cdot g(k_j \oplus y)$) demand:

Bit reversal (g)	: 24 cycles (with assembly coding) .
Exclusive or ($\oplus$)	: 1 cycle.
Multiplication ($\cdot$)	: 7 cycles.

Then each round will need 32 cycles. For the five rounds a total of $5 \cdot 32 = 160$ cycles/block. The block having 8 bytes gives $160 / 8 = 20$ cycles/byte.

Since the bit reversal (g) is a simple bitwise operation, a custom processor could implement it as a native instruction, taking 2 or 3 cycles.

8 Acknowledgments

I would like to thank Paulo S. L. M. Barreto, Daniel Bleichenbacher and Scott Fluhrer, who did valuable comments and suggestions regarding Nimbus cipher.

References

1. E. Biham, "On Matsui's Linear Cryptanalysis" Advances in Cryptology – EUROCRYPT

2. E. Biham, "New Types of Cryptanalytic Attacks Using Related Keys" Journal of Cryptology, v. 7, n. 4, 1994, pp. 229 - 246.
3. Eli Biham, Alex Biryukov, Adi Shamir, "Miss in the Middle Attacks on IDEA and Khufu". Fast Software Encryption 1999: 124-138
4. E. Biham, Vladimir Furman, "Impossible Differential of 8-Round Mars' Core"
5. E. Biham, "New Types of Cryptanalytic Attacks Using Related Keys", in Journal of Cryptology v 7 (1994) no 4 pp 229-246
6. T. Jakobsen and L. R. Knudsen, "The interpolation attack on block ciphers", Fast Software Encryption, LNCS 1267, E. Biham, Ed., Springer-Verlag, 1997, pp. 28–40.
7. L. R. Knudsen, "Truncated and Higher Order Differentials", Fast Software Encryption, 2nd International Workshop Proceedings, Springer-Verlag, 1995, pp. 196 - 211.
8. P. C. Kocher, J. Jaffe, B. Jun, "Introduction to differential power analysis and related attacks", available from <http://www.cryptography.com/dpa/technical/>.
9. P. C. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems", in Advances in Cryptology - Crypto 96, Springer LNCS, v 1109, pp 104 - 113
10. X. J. Lai, "Higher Order Derivative and Differential Cryptanalysis", private communication, September 30, 1993.
11. S. Lucks, "The Saturation Attack - a Bait for Twofish"
12. A. W. Machado, "A New Algorithm for the Cipher Contest", sci.crypt Usenet posting, 21 Aug 2000.
13. M. Matsui, "Linear Cryptanalysis Method for DES Cipher", in Advances in Cryptology - Eurocrypt 93, Springer LNCS v 765 pp 386 – 397
14. Intel, "Itanium Microprocessor Architecture Reference For Software Optimization", August 2000